Problème du Voyageur de Commerce

Rapport de Projet

Élèves :
Mateo Damiani
Yoen Pesquer

Enseignants :
Mme Carla Selmi
Mr Olivier Mallet

4 janvier 2026

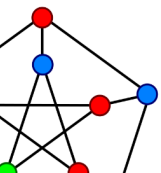


Table des matières

1 Introduction **2**
- **a** Explication du projet . . . 2
- **b** Structure de données utilisées . . . 3
- **c** Initialisation des Eléments . . . 3
  - **I** L'utilisateur donne un fichier valide . . . 3
  - **II** L'utilisateur ne donne pas de fichier . . . 4
- **d** Fonction de Conversion . . . 4

2 Corps du projet **5**
- **a** PPP et OptPPP . . . 5
  - **I** Explication de l'algorithme . . . 5
  - **II** Implémentation . . . 6
  - **III** OptPPP . . . 7
  - **IV** Algorithme de Décroisement . . . 7
- **b** PvcPrim . . . 9
  - **I** Explication de l'algorithme . . . 9
  - **II** Algorithme de Prim . . . 9
  - **III** Parcours Préfixe . . . 10
  - **IV** Combinaison des algorithmes . . . 12
- **c** HDS . . . 13
  - **I** Explication de l'algorithme . . . 13
  - **II** Implémentation . . . 13
- **d** Programme Principal . . . 16

3 Analyse Statistique **17**
- **a** Analyse de complexité . . . 17
  - **I** Présentation des complexités . . . 17
  - **II** Explications des compléxités . . . 17
- **b** Résultats . . . 19

4 Conclusion **20**

1 Introduction

a Explication du projet

Le problème du voyageur de commerce est une modélisation d'un voyageur devant organiser la visite de n villes de manière à minimiser la distance parcourue. Ce problème est NP-Complet et l'objectif de ce projet sera de comparer différents algorithmes de résolutions sur la distance trouvé et sur le temps de calcul. Les villes seront assimilées à des points de la forme (x,y) avec 0 <= x,y <= 1. Le problème sera traité avec 10 points mais les solutions programmées seront compatibles avec un nombre de points arbitraires.

FIGURE 1 – Illustration du PVC, Source : Wikipédia

(Toutes fonctions apparaissant dans le rapport seront décrites, mais les spécifications détaillées sont dans les fichiers du projet).

b Structure de données utilisées

Les différents algorithmes n'utilisent pas les mêmes structures, ainsi le fichier Structure.py contient des fonctions permettant de passer d'une forme à l'autre.

Tous les algorithmes ont besoin d'une liste de points et d'une matrice contenant touts les poids de toutes les arêtes possibles selon cette liste de points.

```
1  :  [0.21893181531451833, 0.6407859868341562]
2  :  [0.3484260989310952, 0.9314596360029103]
3  :  [0.2228783304418851, 0.23883185405642038]
4  :  [0.7425660304973071, 0.23235012425656665]
5  :  [0.31976044715691565, 0.655206241806412]

[0.0, 0.3182136700559081, 0.4019735063891055, 0.6640878293942712, 0.10185458631143993]
[0.3182136700559081, 0.0, 0.7039144028195009, 0.8025586552204326, 0.27773666916113976]
[0.4019735063891055, 0.7039144028195009, 0.0, 0.5197281197030738, 0.42749710561986864]
[0.6640878293942712, 0.8025586552204326, 0.5197281197030738, 0.0, 0.5979731243568872]
[0.10185458631143993, 0.27773666916113976, 0.42749710561986864, 0.5979731243568872, 0.0]
```

FIGURE 2 – Illustration des structures avec la liste de points, puis la matrice de distance.

c Initialisation des Eléments

La liste de points est définit de deux manières selon l'entrée utilisateur :

I L'utilisateur donne un fichier valide

Le fichier a déjà été ouvert au moment de l'exécution et il est convertit grâce à cette fonction.

Convertit le fichier en liste de point

```python
def ConvertirEnPointList(file):
  L = file.readlines()
  file.close()
  res = []
  for i in L:
      i = i.replace('\n','')
      if not(len(i) == 0):
          res.append(ast.literal_eval(i))
  return res
```

II L'utilisateur ne donne pas de fichier

Dans ce cas, il sera demandé à l'utilisateur un nombre de points puis la liste sera initialisé de manière aléatoire.

Créer une liste aléatoire

```
def initPointList(n):
  res = []
  for i in range (n):
      res.append([rd.random(), rd.random()])
  return res
```

Puis la matrice est crée à partir de la liste de points par cette fonction :

Créer la matrice de distance

```
def initD(pointList, n):
  M = np.zeros((n, n))
  for i in range (n):
      for j in range (n):
          M[i][j] = np.linalg.norm(np.array((pointList[i][0],
          ↪  pointList[i][1])) - np.array((pointList[j][0],
          ↪  pointList[j][1])))
  return np.array(M).tolist()
```

d Fonction de Conversion

Parmi les algorithmes travaillés, tous utilisent la matrice de distance mais l'algorithme de Prim utilise une liste de sommets et d'arêtes. Il faut donc les fonctions de conversions appropriées.

Créer la liste d'arêtes

```
def listeEnLA(list, D):
  LA = []
  length = len(list)
  for x in range (length):
      for y in range (x + 1, length):
          LA.append((list[x], list[y], D[x][y]))
  return LA
```

Il y a également les fonctions de conversions d'un circuit ou d'une liste de points en liste de sommets et les fonctions inverses.

2 Corps du projet

Dans cette partie, on va parler des algorithmes un par un de manière détaillé. On commencera par les algorithmes PPP et OptPPP puis PvcPrim et enfin HDS.

a PPP et OptPPP

I Explication de l'algorithme

Le principe du PPP (point le plus proche) consiste à ajouter à un circuit des points un par un, le point ajouté sera le point le plus proche de l'un des points du circuit. Sur cet exemple, les points A et B sont déjà dans le circuit et pour chacun des autres points on va trouver le point le plus proche de l'un des points. Une fois trouvé, on ajoute l'arête de poids minimal au circuit.

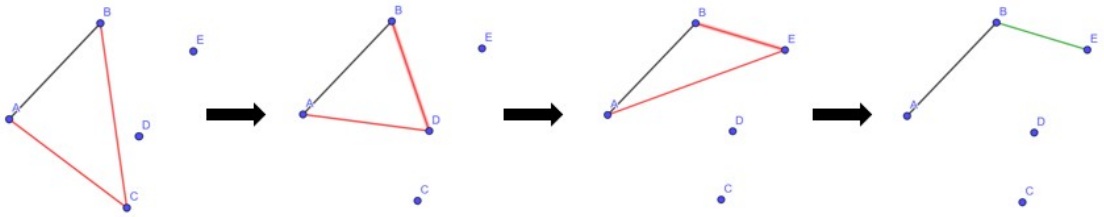


FIGURE 3 – Illustration de l'algorithme PPP.

II Implémentation

L'algorithme a été implémenté de la manière suivante :

Créer la liste d'arêtes

```
# PPP : Renvoie le cycle résultant au PVC et la distance totale parcourue
↪ selon la stratégie du point le plus proche avec une liste de points et
↪ une matrice des distances
Procédure PPP(longueurCircuit : Entier, circuit : Tableau[1..n+1], list :
↪ Tableau[1..n], D : Tableau[1..n] de Tableau[1..n] de réels):
  Var pointList : Tableau[1..n], i, j, index, Q1, minValue, minInsert,
  ↪ indexCycle, indexPoint, indexMin : Entier, res : Tableau[1..n+1],
  ↪ dist, distMin : Réels
  Début
  pointList <- list
  index <- 0
  Q1 <- pointList[0]
  Ajouter(res, Q1)
  RetirerIndex(pointList, index)
  Tant que (Longueur(pointList) != 0)
    Début
    minValue <- 0  #Représente l'indice du point le plus proche du circuit
    ↪ dans pointList
    minInsert <- 0  #Représente l'indice du point du circuit le plus
    ↪ proche de minValue
    Pour i allant de 1 à Longueur(pointList) faire
      Début
      Pour j allant de 1 à Longueur(res) faire
        Début
        indexCycle <- Index(list, res[j])
        indexPoint <- Index(list, pointList[i])  #Récupère les indices
        ↪ selon list (qui reste fixe) afin d'ajouter les bons points au
        ↪ circuit
        indexMin <- Index(list, pointList[minValue])
        dist <- D[indexCycle][indexPoint]
        distMin <- D[indexCycle][indexMin]
        Si distMin > dist faire #On cherche les indices minimums
          Début
          minValue <- i
          minInsert <- j
          FinSi
        FinPour
      FinPour
    Insérer(res, minInsert, pointList[minValue])  #Ajoute le point le plus
    ↪ proche au circuit
```

```
    Retirer(pointList, minValue)  #Le point a été ajouté donc il est
    ↪  retiré de pointList
    FinTantQue
  Inverser(res)  #On ajoute par insertion donc le circuit est inversé
  Ajouter(res, res[0])  #On ajoute le point de départ à la fin afin de
  ↪  créer un vrai circuit
  longueurCircuit <- calculateDistance(D, res, list)
  circuit <- res
  FinProcédure
```

III OptPPP

Le principe d'OptPPP est simple : appliquer un algorithme de décroisement à PPP, c'est-à-dire obtenir un circuit final basé sur le circuit obtenu par PPP sans croisements. Ce décroisement est forcément rentable par inégalité triangulaire (au pire la longueur du circuit ne change pas).

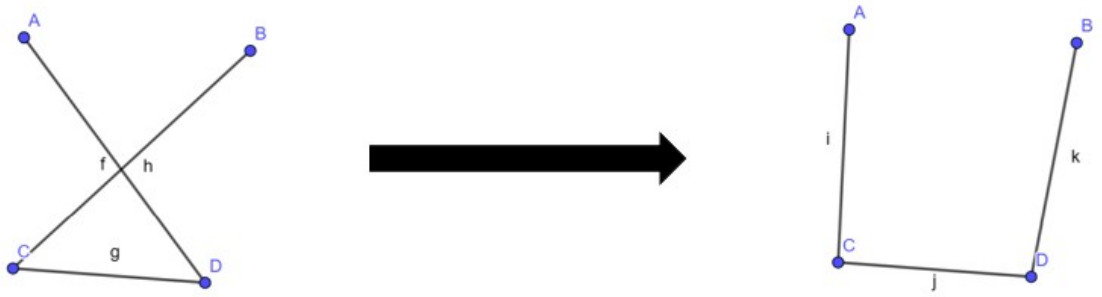


FIGURE 4 – Exemple de décroisement

IV Algorithme de Décroisement

L'objectif est de tester toutes les couples possibles d'arêtes du circuit, puis de vérifier si il n'y a pas de croisement. Pour vérifier si il y a un croisement, il existe plusieurs algorithmes pour déterminer si deux segments se croisent (dans ces algorithmes on considère un segment comme un couple de point AB et CD).

Par exemple, un algorithme possible est d'étendre les segments (transformer en droite) puis d'utiliser les déterminants, en les calculants on peut obtenir deux formules permettants de calculer chacunes des coordonées du point d'intersection des deux droites, ensuite il suffit de vérifier si ce fameux point d'intersection est inclus dans les deux segments. Cet algorithme est néanmoins couteux en calcul et nécessite de réaliser des divisions (menant à des arrondis et donc des imprécisions).

On a donc décider d'utiliser un algorithme plus géométrique basé sur l'orientation des points. Prenons l'exemple de décroisement, on peut remarquer que l'ordre des points dans le chemin est ADCB, réalisons toutes les combinaisons de 3 points conservons l'ordre des points. On obtient donc 4 trios :

- ADC
- ACB
- ADB
- DCB

Maintenant, on va regarder l'orientation de ces points sur la courbe avec ADC. Soit $A = (x_A, y_A), D = (x_D, y_D), C = (x_C, y_C)$ Si on a : $(y_C - y_A) * (x_D - x_A) > (y_C - y_A) * (x_D - x_A)$ Alors les points ADC sont dans le sens antihoraire, on a donc :

- ADC dans le sens horaire
- ACB dans le sens antihoraire
- ADB dans le sens horaire
- DCB dans le sens antihoraire

On remarque que les deux trios ayant le sens opposé sont symétriques par rapport à une droite.

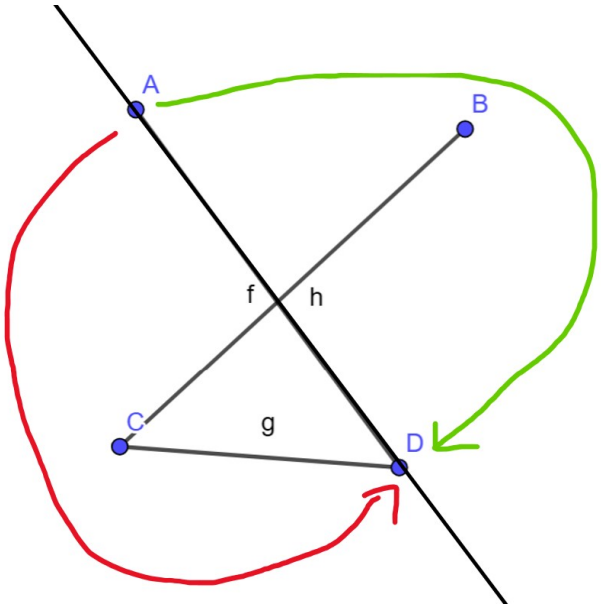


FIGURE 5 – Illustration de symétrie.

Ainsi, si on a $Sens(ADC)! = Sens(ABD) \wedge Sens(ACB)! = Sens(DCB)$
alors il y a un croisement dans le chemin ADCB (ce qui est le cas sur la figure). Le décroisement consiste simplement à échanger les sommets "au milieu" du chemin (ici D et C).

b PvcPrim

I Explication de l'algorithme

Un autre moyen de résoudre le PVC est de passer par l'algorithme de Prim. L'algorithme de Prim renvoie en temps en normal un arbre couvrant minimum (ACM) d'un graphe. Dans le cadre du PVC, on peut transposer les points du plan par un graphe complet dont les poids des arêtes sont stockés dans la matrice de distance. Néanmoins, un ACM n'est pas une solution valide, il faut un circuit. Ainsi, il faut donc réaliser un parcours préfixe de l'ACM afin d'obtenir une solution valide.

II Algorithme de Prim

Pour ce projet, on utlise une version optimisée de l'algorithme utlisant une file de priorité afin d'avoir une complexité plus faible. Ici on travaille donc avec une liste d'arêtes LA et une file de priorité omega contenant les arêtes accessibles depuis un sommet quelconque de l'ACM. Par nature de file de priorité, l'arête en tête de file de omega sera celle avec le poids le plus faible et donc celle offrant une couverture minimale des sommets du graphe. Il est également nécessaire de stocker une liste contenant les sommets déjà ajoutés à l'ACM afin de s'assurer qu'il n'y a pas de doublons dans l'ACM. Voici l'algorithme en langage algorithmique :

Algorithme de Prim efficace

```
# Algo_Prim : renvoie un arbre couvrant de poids minimum selon
↪  l'algorithme de Prim en fonction d'une liste de sommets, une liste
↪  d'aretes et un sommet
Fonction Algo_Prim(S : Tableau[1..n] d'Entiers, LA : Tableau[1..m] d'Arête,
↪  Q : Entier)
  var ACM : Tableau[1..] d'arêtes, T : Tableau [1..] d'Entiers, omega :
  ↪  Tableau[1..m] d'Arête,
  length, x, y, t, s : Entier, poids, pmin : réels, arete, arete_min :
  ↪  Arête
  Début
  T <- [Q]
  length <- n
  TantQue y < length  #Continue tant que tous les points n'ont pas été
  ↪  ajouté à la liste T
    Début
    Pour arete dans LA
      Début
      x, y, poids <- arete
      Si (estDans(T, x) et non(estDans(T, y))) ou (non(estDans(T, x)) et
      ↪  estDans(T, y))  #On ajoute toutes les arêtes accessibles à
      ↪  partir d'un seul point du tas à omega (file de priorité)
        Début
```

```
        AjouterFile(omega, (poids, x, y))
        FinSi
      FinPour
    pmin, x, y <- Défiler(omega)  #On récupère la première arête de la
    ↪  file (celle de poids minimal)
    arete_min <- (pmin, x, y)
    p, t, s <- arete_min
    Si (non(estDans(T, t)))  #On ajoute à T le sommet de l'arête minimale
    ↪  qui n'était pas déjà dans T puis on ajoute l'arête à l'arbre
    ↪  couvrant de poids minimal
      Début
      Ajouter(T, t)
      Ajouter(ACM, (t, s, p))
      FinSi
    Si (non(estDans(T, s)))
      Début
      Ajouter(T, s)
      Ajouter(ACM, (t, s, p))
      FinSi
    FinTantQue
  renvoyer ACM  #On renvoie l'arbre
  Fin
```

L'algorithme renvoie donc un ACM du graphe complet illustrant le PVC.

III Parcours Préfixe

Le but du parcours préfixe est de "transformer" un ACM en un circuit (qui sera une solution du PVC). Pour cela, il faut ajouter les sommets dans le bon ordre. Dans une arête (x, y) on considère x le sommet entrant de l'arête et y le sommet sortant de l'arête.

On parcourt donc toutes les arêtes de l'ACM et si on croise un nouveau sommet entrant, on l'ajoute à la fin du circuit et si on croise un nouveau sommet sortant on l'ajoute au début du circuit (sauf si il s'agit du dernier sommet dans ce cas on l'ajoute à la fin).

Parcours Préfixe

```
# ParcoursPrefixe : renvoie le circuit (et son poids total) crée par un
↪  parcours préfixe d'un arbre couvrant de poids minimal à partir d'une
↪  matrice de distance
Fonction ParcoursPrefixe(ACM : Tableau[1..] d'Arête, D : Tableau[1..n] de
↪  Tableau[1..n] de réels)
  var circuit : Tableau[1..n+1] de Points, x, y, index : Entier, poids,
  ↪  longueurCircuit : réel, arete : Arête
  Début
```

```
Pour arete dans ACM  #On parcourt toutes les arêtes de l'arbre
  Début
  x, y, poids <- arete
  Si (non(estDans(circuit, x)))  #Si on croise un nouveau sommet entrant
  ↪  (l'arête est de la forme x--y), on l'ajoute à la fin du circuit
    Début
    Ajouter(circuit, x)
    FinSi
  index <- Index(circuit, x)
  Si (non(estDans(circuit, y)))  #Si on croise un nouveau sommet sortant,
  ↪  on l'ajoute à la fin du circuit si il s'agit du dernier sommet du
  ↪  circuit ou au tout début sinon.
    Début
    Si index == (Longueur(circuit))
      Début
      Ajouter(circuit, y)
      FinSi
    Sinon
      Début
      Insérer(circuit, y, 1)
      FinSinon
    FinSi
  FinPour
x <- circuit[1]
y <- circuit[Longueur(circuit)]
Ajouter(circuit, circuit[1])  #On ajoute l'arête reliant le premier et
↪  le dernier sommet du circuit
longueurCircuit <- 0  #On calcule le poids du circuit
Pour i allant de 1 à n:
  Début
  longueurCircuit <- longueurCircuit + D[circuit[i]][circuit[i+1]]
  FinPour
renvoyer [longueurCircuit, circuit]  #On renvoie la longueur du circuit
↪  et le circuit en lui-même
Fin
```

IV Combinaison des algorithmes

Enfin, il suffit de combiner les deux algorithmes pour obtenir une solution. On initialise les structures de l'algorithme de Prim (liste de sommets et liste d'arêtes), puis on applique l'algorithme de Prim et on renvoie le résultat du parcours préfixe.

PvcPrim

```
# PvcPrim : renvoie le cycle résultant au PVC selon l'algorithme de Prim
↪  et un parcours préfixe selon une liste de points et une matrice des
↪  distances
Fonction PvcPrim(pointList : Tableau[1..n] de Points, D : Tableau[1..n] de
↪  Tableau[1..n] de réels)
  var S : Tableau[1..n] d'Entiers, l : Tableau[1..] d'Arête, ACM :
  ↪  Tableau[1..] d'Arête
  Début
  S <- pointListEnSommets(pointList)  #On convertit la liste de point en
  ↪  liste de sommets
  l <- listeEnLA(S, D)  #On convertit la matrice de distance en liste
  ↪  d'arêtes
  ACM <- Algo_Prim(S, l, 1)  #On récupère l'arbre résultant de
  ↪  l'algorithme de Prim
  renvoyer ParcoursPrefixe(ACM, D)  #On renvoie le résultat du parcours
  ↪  préfixe
  Fin
```

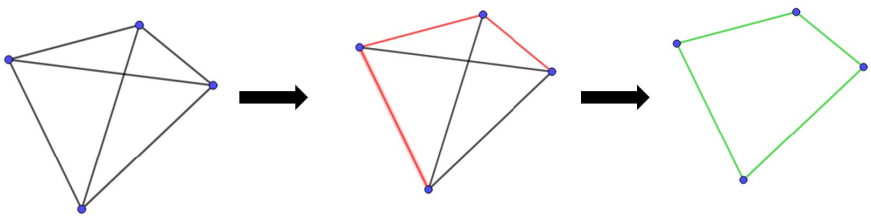


FIGURE 6 – Illustration de l'algorithme PvcPrim en 3 étapes, initialisation, ACM puis circuit.

c HDS

I Explication de l'algorithme

Le principe de l'algorithme HDS est d'énumérer toutes les possibilités puis d'éliminer les solutions non optimales. Il s'agit ici d'un algorithme de backtracking avec comme seule différence que des tests sont réalisés avant un appel récursif afin de déterminer si il est potentiellement intéressant.

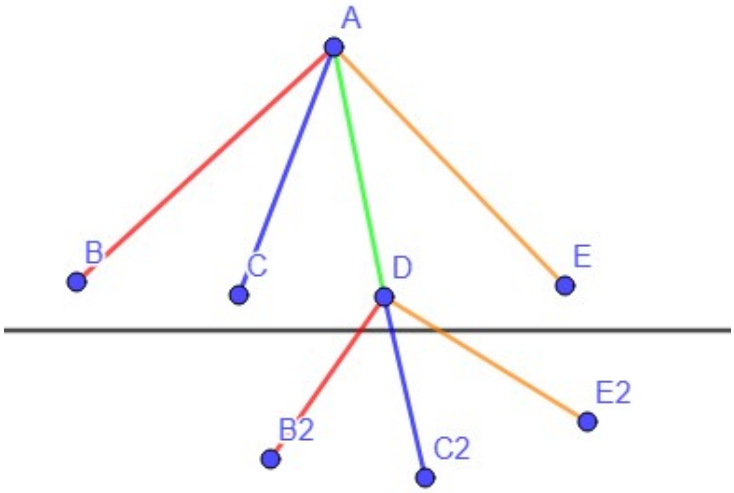


FIGURE 7 – Illustration de l'algorithme HDS, on a un sommet de départ A, après éléminations des options non optimales, on réalise un appel récursif en ayant ajouté D au circuit et on se retrouve de nouveau avec plusieurs options.

II Implémentation

L'algorithme HDS se divise en 3 parties, une initialisation, un calcul récursif puis un décroisement car l'algorithme peut renvoyer un circuit contenant des croisements.

L'initialisation consiste en la création de plusieurs tableaux pour stocker notamment : les sommets déjà visités, le chemin courant et le circuit final. Il y a également un calcul de la borne en utilisant l'heuristique de la demi somme. Elle consiste à calculer la somme des deux arêtes de poids minimales de chaque sommet et de diviser cette somme par deux. On obtient donc une borne correspondant au plus petit poids d'un cycle hamiltonien contenant tous les sommets ainsi si durant le calcul récursif on obtient une longueur de circuit supérieure à cette borne, alors ce circuit ne peut pas être une bonne solution et on "stoppe" le calcul récursif pour ce chemin.

Le calcul récursif se sépare en deux parties, le cas d'arrêt et le calcul.

HDS : Cas d'arrêt

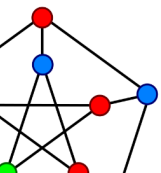
```
Si hauteur == n  #Cas d'arrêt de la récursivité
  Début
  Si D[chemin[hauteur]][chemin[0]] != 0  #On vérifie que le chemin n'est
  ↪  pas une boucle
    Début
    res <- poids + D[chemin[hauteur]][chemin[0]]  #On met à jour le poids
    ↪  afin d'ajouter la dernière arête
    Si res < longueurCircuit  #Si l'ajout de la dernière arête est
    ↪  avantageux alors on contruit le circuit et on le renvoie
      Début
      circuit[:n + 1] <- chemin[:]  #On remplace les n premiers éléments
      ↪  par chemin
      circuit[n] <- chemin[0]
      longueurCircuit <- res
      FinSi
    FinSi
    renvoyer longueurCircuit, circuit  #On renvoie la longueur du circuit
    ↪  et le circuit en lui-même
  FinSi
```

Le cas d'arrêt est atteint si la hauteur du chemin est égale au nombre de points (<=> le chemin contient tous les points), c'est à ce moment que le circuit final est crée et que les résultats sont renvoyés à l'utilisateur.

La phase de calcul consiste à boucler sur tous les sommets, puis pour chaque sommet, on vérifie si il a déjà été visité. Si il n'a pas été visité, on calcule une nouvelle borne via l'heuristique de la demi somme et enfin on vérifie si la nouvelle borne plus le poids de l'arête (celle entre le dernier sommet de la branche et celui sur lequel on travaille dans la boucle) est inférieur à la longueur d'une solution déjà trouvé (donc si ce nouveau chemin est avantageux par rapport à une solution précédente). Si toutes ces conditions sont réunis, on lance un appel récursif en ajoutant le sommet dans le chemin, on recommence donc l'algorithme mais avec un sommet supplémentaire dans le chemin (on se rapproche donc du cas d'arrêt assurant une fin à l'algorithme).

HDS : Calcul

```
Pour i allant de 1 à n  #On boucle sur tous les sommets
  Début
  Si D[chemin[hauteur]][chemin[0]] != 0 et non(visited[i])  #On vérifie si
  ↪  le sommet i est relié au dernier sommet du chemin et si il n'a pas
  ↪  déjà été visité
    Début
```



```
    branche_temp <- branche  #On sauvegarde la branche pour pouvoir la
    ↪  modifier
    poids <- poids + D[chemin[hauteur]][i]  #On suppose que i est le
    ↪  prochain sommet du circuit et donc on ajoute le poids de l'arête
    curr_min1er, curr_min2nd <- demiSomme(D, chemin[hauteur])  #On cherche
    ↪  le 1er et le 2eme poids minimum du dernier sommet du chemin
    min1er, _ <- demiSomme(D, i)  #On cherche le 1er et le 2eme poids
    ↪  minimum du sommet i
    Si hauteur == 1  #Si la hauteur de la branche vaut 1 alors on
    ↪  soustrait la moyenne des premiers minimums
      Début
      branche <- branche - ((curr_min1er + min1er) / 2)
      FinSi
    Sinon  #Sinon il y a au moins 2 points donc on soustrait la moyenne du
    ↪  1er minimum du sommet du chemin et le 2ème minimum de i
      Début
      branche <- branche - ((curr_min2nd + min1er) / 2)
      FinSinon
    Si branche + poids < longueurCircuit  #Appel récursif si ajouter le
    ↪  sommet i au chemin est avantageux par rapport au circuit courant
      Début
      chemin[hauteur] <- i
      visited[i] <- Vrai  #Mise à jour du chemin et du tableau visited
      longueurCircuit, circuit <- HDSCalcul(D, branche, poids, hauteur + 1,
      ↪  chemin, visited, longueurCircuit, circuit)  #Appel récursif = on
      ↪  avance dans la recherche d'une solution
      FinSi
    poids <- poids - D[chemin[hauteur]][i]  #On prépare la boucle suivante
    ↪  en annulant les mises à jours ayant eu lieu durant la boucle
    branche <- branche_temp
    MettreAFaux(visited)
    Pour j allant de 1 à hauteur  #On redéfinit le tableau visited afin
    ↪  d'assurer de ne pas passer des circuit possibles
      Début
      visited[chemin[j]] <- Vrai
      FinPour
    FinSi
  FinPour
```

Une fois que les calculs ont été réalisés, on décroise le résultat afin de limiter de s'assurer d'avoir une solution optimale.

d Programme Principal

Le programme principal a pour objectif d'initialiser la liste de points et d'exécuter les bons tests selon les actions de l'utilisateur. Il peut notamment insérer un fichier de points (pour avoir des points personnalisés) ou générer des points pseudo-aléatoires, il a également le choix sur le nombre de fois où les tests seront réalisés. Si l'utilisateur ne souhaite qu'un seul essai, il aura des informations détaillés sur les résultats des algorithmes tandis que si il choisit plusieurs essais alors il aura des moyennes sur les temps d'exécution et les distances des circuit obtenus par les différents algorithmes.

```
Enter the path of a file or none for random points: none
Enter a number of Points: 10
How many tries do you want ? 100

Résultats de 100 essais avec 10 points

Résultat du PPP :
Temps Moyen : 5.467653274536133e-05
Distance Moyenne : 3.9911466961653814

Résultat du OPTPPP :
Temps Moyen : 8.30531120300293e-05
Distance Moyenne : 3.599052525386563

Résultat de l'algorithme de Prim :
Temps Moyen : 0.0006920266151428223
Distance Moyenne : 3.5950505398886587

Résultat du HDS :
Temps Moyen : 0.00798452615737915
Distance Moyenne : 3.0314005208015287

Le circuit du HDS est 24.04687796331713 pourcents plus court que celui du PPP
Le temps d'exécution du HDS est 146.03204988444597 fois plus long que PPP

Le circuit du HDS est 15.772262299063414 pourcents plus court que celui du OptPPP
Le temps d'exécution du HDS est 96.1375915027989 fois plus long que OptPPP

Le circuit du HDS est 15.67850056162456 pourcents plus court que celui du Prim
Le temps d'exécution du HDS est 11.537888836444942 fois plus long que Prim
```

FIGURE 8 – Illustration du programme principal après exécution.

3 Analyse Statistique

Maintenant, on va comparer les différents algorithmes par rapport à HDS (qui offre une solution optimale) afin de savoir quel est le meilleur algorithme en terme de distance obtenu mais aussi en terme de temps d'éxécution. Mais pour cela il faut connaître les complexités des algorithmes.

a Analyse de complexité

I Présentation des complexités

Voici les complexités des algorithmes avec n le nombre d'éléments de la liste de points :

Complexité : Comparaisons des algorithmes :

PPP : $O(n^2)$
OptPPP : $O(2 * n^2)$
PvcPrim : $O(m * log2(n) + n * log2(n) + n)$
HDS : $O(n!)$ (dans le pire des cas)

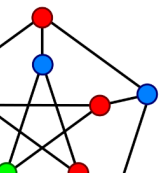
II Explications des compléxités

Pour PPP, le programme est composé de deux boucles for sur la longueur de la liste de points et du circuit résultant, elle même dans une boucle while attendant que la longueur de la liste de points soit égale à 0. A chaque itération de la boucle while, la longueur de la liste de points diminue de 1 tandis que celle du circuit augmente de 1. Ainsi, les deux boucles for ne font factuellement que n étapes dans une boucle while réalisant n boucles. Soit donc une complexité de $O(n^2)$.

Pour OptPPP, la procédure de décroisement possède une complexité de $O(n^2)$ et il réalise un appel à l'algorithme PPP. On obtient donc une complexité de $O(2 * n^2)$.

Pour PvcPrim, il faut additioner la complexité de l'algorithme de Prim et celle du parcours préfixe. L'algorithme de Prim possède comme complexité : $O(m * log2(n) + n * log2(n))$ car il s'agit de celle donnée dans le cours avec l'implantation efficace. Le parcours préfixe possède une complexité linéaire ($O(n)$).

Et enfin pour HDS, il est nécessaire de déterminer la complexité de la partie calculatoire de l'algorithme. Pour calculer, on réalise une boucle sur tous les sommets et on réalise (potentiellement) un appel récursif sur chaque sommet qui n'a pas déjà été visité. Donc dans le pire des cas, le nombre d'appel récursif réalisé est n - h avec h le nombre de sommets visité au moment de la boucle, l'algorithme est équivalent à un algorithme de backtracking. En prenant l'exemple de cette illustration, on voit la complexité de l'algorithme.



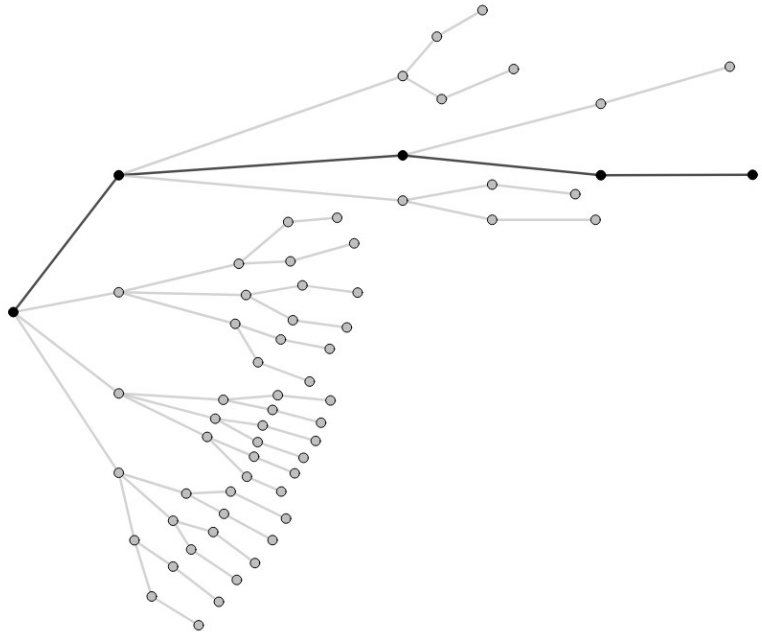


FIGURE 9 – Illustration de l'exécution de l'algorithme dans le pire des cas avec 5 points.

Néanmoins, ce cas n'est pas réaliste car grâce à l'heuristique de la demi-Somme, on se base sur le plus petit poids d'un cycle hamiltoniene et cela élimine environ la moitié des appels récursifs. En pratique, la complexité est donc plus proche de $O(n!)/2$.

b Résultats

Les chiffres si dessous sont une moyenne des temps d'exécutions et des longueurs de circuit obtenus par les algorithmes réalisés avec 10 points. 1000 essais ont eu lieu afin d'obtenir des chiffres pertinents.

Temps d'exécution et longueur de circuit : Comparaisons des algorithmes :

PPP :

- Temps : 5.201840400695801e-05 s
- Distance : 3.9842512880562255

OptPPP :

- Temps : 7.856988906860351e-05 s
- Distance : 3.5914547990521286

PvcPrim :

- Temps : 6.747841835021973e-04 s
- Distance : 3.5737014736900448

HDS :

- Temps : 7.396111249923706e-03 s
- Distance : 3.0370304258616194

Grâce à ces chiffres, on peut remarquer que les algorithmes PPP et OptPPP sont très rapides mais offrent un résultat approximatif par rapport aux autres algorithmes. PvcPrim offre un résultat similaire à OptPPP malgré le fait qu'il soit environ 10 fois plus long tandis que HDS offre de loin le meilleur résultat (attendu car il est censé renvoyer le résultat exact) mais il est encore 10 fois long environ que PvcPrim (100 fois plus long que OptPPP). Ainsi, le "meilleur" algorithme pour résoudre le PVC dépend des contraintes sur la solution recherché, le HDS sera meilleur pour trouvé une solution précise mais le coût en temps n'est pas forcément envisageable. Si on cherche un temps d'exécution court, alors OptPPP semble être le meilleur algorithme. Néanmoins, PvcPrim possède une complexité logarithmique, donc plus le nombre de points est grand, plus le gain en temps est intéressant par rapport à OptPPP.

FIGURE 10 – Illustration comparant les temps d'exécutions entre OptPPP et PvcPrim.

4 Conclusion

Pour conclure, il est intéressant de travailler sur plusieurs algorithmes afin de résoudre le même problème. Ce projet nous a permis de voir plusieurs approches de résolutions et d'illustrer le fait que tous les algorithmes ont des points positifs et négatifs. Au final, il faut réaliser un choix entre qualité du résultat et temps d'exécution et l'un sera préférable à l'autre selon les besoins.